

1. Deep-Dive Mathematical Derivation from First Principles

In Nomura's Cash Equities Central Risk Book (CRB) team, managing a massive inventory of continuous client order flows means you are exposed to significant tail risk. Assuming asset prices behave like a continuous Geometric Brownian Motion (GBM) can be dangerous. A continuous model cannot capture sudden jumps caused by block liquidations or surprise institutional flow.

Merton's (1976) Jump-Diffusion model extends the continuous asset framework by adding a discontinuous compound Poisson process. This allows the model to capture discrete price drops while maintaining continuous Brownian updates for normal trading environments.

Phase A: Line-by-Line Breakdown of the Merton Stochastic Differential Equation (SDE)

The asset price process S_t satisfies the following linear stochastic differential equation:

$$\frac{dS_t}{S_{t-}} = (\mu - \lambda\kappa)dt + \sigma dW_t + (J_t - 1)dN_t$$

Where:

- $S_{t-} = \lim_{s \uparrow t} S_s$ establishes the asset price immediately *before* any jump occurs at time t .
- $\mu \in \mathbb{R}$ is the continuous drift coefficient.
- $\sigma \in \mathbb{R}^+$ is the continuous volatility diffusion coefficient.
- W_t is a standard Brownian motion satisfying $\mathbb{E}[dW_t] = 0$ and $\mathbb{E}[dW_t^2] = dt$.
- N_t is an independent homogeneous Poisson process with constant intensity parameter $\lambda > 0$, meaning $\mathbb{P}(dN_t = 1) = \lambda dt + o(dt)$.
- J_t is a positive random variable defining the absolute jump size multiplier when a jump occurs at time t . Merton parameterizes log-jump sizes as normally distributed: $\ln J_t \sim \mathcal{N}(\mu_J, \delta^2)$.
- $\kappa = \mathbb{E}[J_t - 1] = \mathbb{E}[J_t] - 1$ is the expected relative jump size. Since $\ln J_t \sim \mathcal{N}(\mu_J, \delta^2)$, the log-normal expectation identity yields $\mathbb{E}[J_t] = \exp(\mu_J + \frac{1}{2}\delta^2)$. Thus, $\kappa = \exp(\mu_J + \frac{1}{2}\delta^2) - 1$.
- The compensated drift term $-\lambda\kappa dt$ is added to ensure that if we want the total expected asset price return to equal μdt , the jump process must be centered around its expectation: $\mathbb{E}[(J_t - 1)dN_t] = \mathbb{E}[J_t - 1]\mathbb{E}[dN_t] = \kappa\lambda dt$.

Phase B: Line-by-Line Derivation of the Log-Return Process via Itô's Lemma for Jump-Diffusions

To derive the discrete log-return distribution over the finite timeline window $[0, T]$, we transform the variable using $f(S_t) = \ln S_t$. Because the process is discontinuous, we must apply the generalized Itô's Lemma for jump-diffusions:

$$df(S_t) = \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S}dS_t^c + \frac{1}{2}\frac{\partial^2 f}{\partial S^2}(dS_t^c)^2 + [f(S_t) - f(S_{t-})]dN_t$$

Where dS_t^c represents the purely continuous parts of the SDE:

$$dS_t^c = (\mu - \lambda\kappa)S_t dt + \sigma S_t dW_t$$

Step 1: Evaluating Continuous Partial Derivatives

$$\frac{\partial f}{\partial t} = 0, \quad \frac{\partial f}{\partial S} = \frac{1}{S_t}, \quad \frac{\partial^2 f}{\partial S^2} = -\frac{1}{S_t^2}$$

Step 2: Evaluating the Continuous Quadratic Variation

Using standard box algebra rules ($dt \cdot dt = 0$, $dt \cdot dW_t = 0$, $dW_t \cdot dW_t = dt$):

$$(dS_t^c)^2 = ((\mu - \lambda\kappa)S_t dt + \sigma S_t dW_t)^2 = \sigma^2 S_t^2 dW_t^2 = \sigma^2 S_t^2 dt$$

Step 3: Evaluating the Discrete Jump Term

When a jump occurs, $S_t = J_t S_{t-}$. The discrete shift is:

$$f(S_t) - f(S_{t-}) = \ln(J_t S_{t-}) - \ln(S_{t-}) = \ln J_t + \ln(S_{t-}) - \ln(S_{t-}) = \ln J_t$$

Step 4: Substituting into the Jump-Diffusion Itô Framework

Substitute these parts back into the generalized equation:

$$d(\ln S_t) = 0 + \frac{1}{S_{t-}} [(\mu - \lambda\kappa)S_{t-} dt + \sigma S_{t-} dW_t] + \frac{1}{2} \left(-\frac{1}{S_{t-}^2} \right) [\sigma^2 S_{t-}^2 dt] + (\ln J_t) dN_t$$

Distribute across the brackets:

$$d(\ln S_t) = (\mu - \lambda\kappa)dt + \sigma dW_t - \frac{1}{2}\sigma^2 dt + (\ln J_t)dN_t$$

Grouping the continuous deterministic dt components yields:

$$d(\ln S_t) = \left(\mu - \frac{1}{2}\sigma^2 - \lambda\kappa \right) dt + \sigma dW_t + (\ln J_t)dN_t$$

Integrating this continuous differential relationship from $t = 0$ to T gives the log-price equation:

$$\ln \left(\frac{S_T}{S_0} \right) = \left(\mu - \frac{1}{2}\sigma^2 - \lambda\kappa \right) T + \sigma W_T + \sum_{j=1}^{N_T} \ln J_j$$

Phase C: Painstaking Step-by-Step Derivation of Expected Value and Total Variance

Let $X_T = \ln \left(\frac{S_T}{S_0} \right)$. We apply the **Law of Total Expectation** and the **Law of Total Variance** by conditioning on the total number of jumps $N_T = n$ over the interval.

Step 1: Analytical Derivation of Expected Return $\mathbb{E}[X_T]$

$$\mathbb{E}[X_T] = \mathbb{E} \left[\left(\mu - \frac{1}{2}\sigma^2 - \lambda\kappa \right) T + \sigma W_T + \sum_{j=1}^{N_T} \ln J_j \right]$$

Since the expectations decouple linearly:

$$\mathbb{E}[X_T] = \left(\mu - \frac{1}{2}\sigma^2 - \lambda\kappa \right) T + \sigma\mathbb{E}[W_T] + \mathbb{E} \left[\sum_{j=1}^{N_T} \ln J_j \right]$$

We know $\mathbb{E}[W_T] = 0$. By Wald's Identity for a compound Poisson process, the expected value of the jump sum is the expected number of jumps multiplied by the expected jump size: $\mathbb{E}[N_T]\mathbb{E}[\ln J_j] = (\lambda T)\mu_J$.

$$\mathbb{E}[X_T] = \left(\mu - \frac{1}{2}\sigma^2 - \lambda\kappa \right) T + \lambda T \mu_J$$

Step 2: Step-by-Step Derivation of Variance $\text{Var}(X_T)$

By the Law of Total Variance:

$$\text{Var}(X_T) = \mathbb{E}[\text{Var}(X_T | N_T)] + \text{Var}(\mathbb{E}[X_T | N_T])$$

Let us calculate each conditional part carefully:

Part A: Evaluating the Expected Conditional Variance $\mathbb{E}[\text{Var}(X_T | N_T)]$

Conditioned on a fixed number of jumps $N_T = n$, the term $\left(\mu - \frac{1}{2}\sigma^2 - \lambda\kappa \right) T$ becomes a non-random constant, meaning its variance contribution drops to 0. Since the Brownian motion and the jump sizes are independent, their variances sum linearly:

$$\text{Var}(X_T | N_T = n) = \text{Var}(\sigma W_T) + \text{Var} \left(\sum_{j=1}^n \ln J_j \right) = \sigma^2 T + n\delta^2$$

Now, take the expectation of this conditional variance over all possible jump counts N_T :

$$\mathbb{E}[\text{Var}(X_T | N_T)] = \mathbb{E}[\sigma^2 T + N_T \delta^2] = \sigma^2 T + \delta^2 \mathbb{E}[N_T] = \sigma^2 T + \lambda T \delta^2$$

Part B: Evaluating the Variance of Conditional Expectations $\text{Var}(\mathbb{E}[X_T | N_T])$

First, compute the expected value of X_T conditioned on a fixed jump count $N_T = n$:

$$\mathbb{E}[X_T | N_T = n] = \left(\mu - \frac{1}{2}\sigma^2 - \lambda\kappa \right) T + \sigma\mathbb{E}[W_T] + \sum_{j=1}^n \mathbb{E}[\ln J_j]$$

$$\mathbb{E}[X_T | N_T = n] = \left(\mu - \frac{1}{2}\sigma^2 - \lambda\kappa \right) T + 0 + n\mu_J$$

Treating this conditional expectation as a random variable dependent on N_T :

$$\mathbb{E}[X_T | N_T] = \left(\mu - \frac{1}{2}\sigma^2 - \lambda\kappa \right) T + \mu_J N_T$$

Now, calculate its variance. The deterministic time components drop out, leaving only the variance of the Poisson process multiplied by the squared scalar coefficient μ_J^2 :

$$\text{Var}(\mathbb{E}[X_T | N_T]) = \text{Var} \left(\left(\mu - \frac{1}{2}\sigma^2 - \lambda\kappa \right) T + \mu_J N_T \right) = \mu_J^2 \text{Var}(N_T)$$

Since the variance of a Poisson process equals its mean ($\text{Var}(N_T) = \lambda T$):

$$\text{Var}(\mathbb{E}[X_T | N_T]) = \mu_J^2 \lambda T$$

Part C: Combining the Results

Summing the results from Part A and Part B yields the total variance formula:

$$\text{Var}(X_T) = (\sigma^2 T + \lambda T \delta^2) + \mu_J^2 \lambda T = \sigma^2 T + \lambda T (\delta^2 + \mu_J^2)$$

This derivation shows how jumps increase the asset's return variance over time. The variance contribution scales linearly with the jump arrival intensity λ and depends on both the variance (δ^2) and the mean magnitude (μ_J^2) of the jumps.

Phase D: First-Principles Derivation of the Characteristic Function $\phi_T(u)$

The characteristic function $\phi_T(u)$ is defined as the expected value of the complex exponentiated return process:

$$\phi_T(u) = \mathbb{E}[e^{iuX_T}] = \mathbb{E}\left[\exp\left(iu\left[\left(\mu - \frac{1}{2}\sigma^2 - \lambda\kappa\right)T + \sigma W_T + \sum_{j=1}^{N_T} \ln J_j\right]\right)\right]$$

We separate the deterministic components from the independent continuous and jump expectations:

$$\phi_T(u) = \exp\left(iu\left(\mu - \frac{1}{2}\sigma^2 - \lambda\kappa\right)T\right) \cdot \mathbb{E}[e^{iu\sigma W_T}] \cdot \mathbb{E}\left[\exp\left(iu\sum_{j=1}^{N_T} \ln J_j\right)\right]$$

Step 1: Evaluating the Continuous Brownian Component Expectation

Since $W_T \sim \mathcal{N}(0, T)$, then $\sigma W_T \sim \mathcal{N}(0, \sigma^2 T)$. Using the standard normal characteristic identity $\mathbb{E}[e^{iuZ}] = \exp\left(-\frac{1}{2}u^2 \text{Var}(Z)\right)$:

$$\mathbb{E}[e^{iu\sigma W_T}] = \exp\left(-\frac{1}{2}\sigma^2 u^2 T\right)$$

Step 2: Evaluating the Discontinuous Jump Component Expectation via the Law of Total Expectation

We evaluate the jump sum by conditioning on the total jump count N_T :

$$\mathbb{E}\left[\exp\left(iu\sum_{j=1}^{N_T} \ln J_j\right)\right] = \mathbb{E}\left[\mathbb{E}\left[\exp\left(iu\sum_{j=1}^n \ln J_j\right) \middle| N_T = n\right]\right]$$

Since the individual jump sizes $\ln J_j \sim \mathcal{N}(\mu_J, \delta^2)$ are independent and identically distributed, their joint characteristic function factors into a product of identical individual expectations:

$$\mathbb{E}\left[\exp\left(iu\sum_{j=1}^n \ln J_j\right)\right] = \prod_{j=1}^n \mathbb{E}[e^{iu \ln J_j}] = (\mathbb{E}[e^{iu \ln J_1}])^n$$

Using the normal characteristic function formula for $\ln J_1$:

$$\mathbb{E} \left[e^{iu \ln J_1} \right] = \exp \left(iu\mu_J - \frac{1}{2}\delta^2 u^2 \right)$$

Substitute this back into the conditional expectation term:

$$\mathbb{E} \left[\exp \left(iu \sum_{j=1}^{N_T} \ln J_j \right) \right] = \mathbb{E} \left[\left(\exp \left(iu\mu_J - \frac{1}{2}\delta^2 u^2 \right) \right)^{N_T} \right] = \mathbb{E} \left[e^{N_T \cdot (iu\mu_J - \frac{1}{2}\delta^2 u^2)} \right]$$

This expression matches the moment-generating function of a Poisson random variable, defined as $\mathbb{E}[e^{N_T z}] = \exp(\lambda T(e^z - 1))$. Setting $z = iu\mu_J - \frac{1}{2}\delta^2 u^2$ yields:

$$\mathbb{E} \left[\exp \left(iu \sum_{j=1}^{N_T} \ln J_j \right) \right] = \exp \left(\lambda T \left(\exp \left(iu\mu_J - \frac{1}{2}\delta^2 u^2 \right) - 1 \right) \right)$$

Step 3: Combining All Terms

Multiplying the continuous and discontinuous components together gives the final **Merton Jump-Diffusion Characteristic Function**:

$$\phi_T(u) = \exp \left(iu \left(\mu - \frac{1}{2}\sigma^2 - \lambda\kappa \right) T - \frac{1}{2}\sigma^2 u^2 T + \lambda T \left(\exp \left(iu\mu_J - \frac{1}{2}\delta^2 u^2 \right) - 1 \right) \right)$$

This analytical function allows you to implement fast option pricing via Fourier transformations (such as the Carr-Madan method) or recover the exact return distribution needed for Value-at-Risk calculations.

Phase E: Structural Impact on Value-at-Risk (VaR) Calibrations

Under a pure Geometric Brownian Motion framework, asset returns are assumed to be normally distributed. This means short-term tail risk like a 99% Value-at-Risk (VaR) can be calculated using a simple linear quantile scaling factor:

$$\text{VaR}_{99\%}^{\text{GBM}} = - \left(\mu - \frac{1}{2}\sigma^2 \right) T + z_{0.99}\sigma\sqrt{T} \quad (\text{where } z_{0.99} = 2.33)$$

In a jump-diffusion framework, the asset return distribution is modeled as a probability-weighted mixture of normal distributions, indexed by the number of realized jumps n :

$$\mathbb{P} \left(\ln \left(\frac{S_T}{S_0} \right) \leq x \right) = \sum_{n=0}^{\infty} \frac{e^{-\lambda T} (\lambda T)^n}{n!} \Phi \left(\frac{x - \left(\mu - \frac{1}{2}\sigma^2 - \lambda\kappa \right) T - n\mu_J}{\sqrt{\sigma^2 T + n\delta^2}} \right)$$

This structural difference has significant implications for risk management within a Central Risk Book:

- 1. Underestimating Short-Term Tail Risk:** Over a short trading horizon (such as a 1-day holding period, $T = \frac{1}{252}$), the diffusion component scales with $\sqrt{T} \approx 0.063$, while the jump probability scales linearly with $T \approx 0.0039$. In a pure diffusion model, the probability of an extreme price drop (e.g., a -5% move) decreases rapidly as the time horizon shrinks. In contrast, a jump-diffusion model accounts for the possibility that a large, discrete jump can occur at any time, even over very short horizons.
- 2. Severe Under-Hedging of Portfolio Out-of-the-Money Deltas:** Because a pure diffusion model understates the probability of large price moves, it assigns a near-zero value to deeply out-of-the-

money options. If the central risk book relies on these continuous approximations during market sell-offs, it will under-hedge its tail risk exposures, leading to unexpected losses when discrete jumps occur.

2. Quantitative Infrastructure & Application of Theory

The following complete, self-contained Python script implements a Merton Jump-Diffusion simulation and risk accounting engine. It avoids high-level wrapper libraries, tracking paths from first principles using vector math. It evaluates the exact returns, computes empirical quantiles for comparison against pure diffusion models, and renders an interactive multi-panel validation dashboard using Plotly.

```
import numpy as np
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# =====
# 1. PRODUCTION JUMP-DIFFUSION MONTE CARLO SIMULATION ENGINE
# =====
def simulate_merton_jump_diffusion(
    S0: float = 100.0, mu: float = 0.05, sigma: float = 0.20,
    lam: float = 2.0, mu_J: float = -0.15, delta: float = 0.05,
    T: float = 1.0, n_steps: int = 252, n_paths: int = 5000, seed: int = 42
) -> tuple:
    """
    Simulates asset price paths under Merton's Jump Diffusion model.
    Uses exact vector constructs to handle continuous drift and compound Poisson
    jumps.
    """
    np.random.seed(seed)
    dt = T / n_steps

    # Calculate the expected relative jump size (kappa)
    kappa = np.exp(mu_J + 0.5 * delta**2) - 1.0

    # Allocate coordinate memory for price paths
    paths = np.zeros((n_steps + 1, n_paths))
    paths[0, :] = S0

    # Pre-generate standard normal innovations for the continuous diffusion
    component
    Z_continuous = np.random.normal(0, 1, size=(n_steps, n_paths))

    # Process steps sequentially over time
    for t in range(1, n_steps + 1):
        # Sample Poisson jump counts for the current time step dt
        n_jumps = np.random.poisson(lam * dt, size=n_paths)

        # Accumulate the log jump sizes for paths where jumps occurred
        log_jump_sum = np.zeros(n_paths)
        for path_idx in range(n_paths):
            if n_jumps[path_idx] > 0:
```

```

        # Sum individual independent normal log-jump sizes
        log_jump_sum[path_idx] = np.sum(np.random.normal(mu_J, delta,
size=n_jumps[path_idx]))

    # Apply the derived log-return updates step-by-step
    continuous_drift = (mu - 0.5 * sigma**2 - lam * kappa) * dt
    diffusion_noise = sigma * np.sqrt(dt) * Z_continuous[t-1, :]

    # Advance the price path
    paths[t, :] = paths[t-1, :] * np.exp(continuous_drift + diffusion_noise +
log_jump_sum)

# Calculate the total log-returns over the entire horizon T
log_returns = np.log(paths[-1, :] / S0)

return paths, log_returns, kappa

# =====
# 2. SEAMLESS COMPARATIVE VALUE-AT-RISK ANALYSIS PIPELINE
# =====
def compute_comparative_var_metrics(jd_returns: np.ndarray, mu: float, sigma:
float, T: float) -> dict:
    """
    Compares the historical quantiles of the Jump-Diffusion simulation
    against the analytical benchmarks of a pure continuous GBM process.
    """
    # Calculate the empirical quantiles from the simulated Jump-Diffusion returns
    var_95_jd = -np.quantile(jd_returns, 0.05)
    var_99_jd = -np.quantile(jd_returns, 0.01)

    # Compute the analytical solutions for a pure continuous GBM process
    gbm_mean = (mu - 0.5 * sigma**2) * T
    gbm_std = sigma * np.sqrt(T)

    var_95_gbm = -(gbm_mean - 1.645 * gbm_std)
    var_99_gbm = -(gbm_mean - 2.326 * gbm_std)

    return {
        'GBM_VaR_95': var_95_gbm, 'JD_VaR_95': var_95_jd,
        'GBM_VaR_99': var_99_gbm, 'JD_VaR_99': var_99_jd
    }

# =====
# 3. INTERACTIVE PLATFORM PLOTLY DASHBOARD GENERATOR
# =====
def generate_merton_risk_dashboard(time_grid: np.ndarray, paths: np.ndarray,
jd_returns: np.ndarray, metrics: dict):
    """
    Renders an interactive dashboard showing asset price path trajectories,
    return distributions, and a comparison of VaR metrics.
    """
    fig = make_subplots(
        rows=2, cols=2,
        subplot_titles=(

```

```

        "Merton Jump-Diffusion Simulated Path Trajectories (Subset)",
        "Log-Return Distribution Tails (Fattened Jump Mixture)",
        "Value-At-Risk (VaR) Discrepancy Matrix",
        "Quantile Loss Under-Estimation Delta"
    ),
    vertical_spacing=0.15, horizontal_spacing=0.12
)

# Panel 1: Trace Asset Price Sample Paths
for idx in range(min(15, paths.shape)):
    fig.add_trace(go.Scatter(
        x=time_grid, y=paths[:, idx], mode='lines',
        line=dict(width=1.5), opacity=0.7, name=f'Path {idx}'
    ), row=1, col=1)

# Panel 2: Log-Return Distribution Histogram
fig.add_trace(go.Histogram(
    x=jd_returns, nbinsx=60, histnorm='probability density',
    marker_color='mediumpurple', opacity=0.75, name='Jump-Diffusion'
), row=1, col=2)

# Superimpose normal reference distribution curve to illustrate tail fattening
x_range = np.linspace(jd_returns.min(), jd_returns.max(), 200)
empirical_mean = np.mean(jd_returns)
empirical_std = np.std(jd_returns)
normal_density = (1.0 / (empirical_std * np.sqrt(2 * np.pi))) * np.exp(-0.5 *
((x_range - empirical_mean) / empirical_std)**2)
fig.add_trace(go.Scatter(x=x_range, y=normal_density, mode='lines',
line=dict(color='lightgreen', width=2.5), name='Normal Fit Curve'), row=1, col=2)

# Panel 3: Value-at-Risk Metric Comparisons
categories = ['95% Confidence VaR', '99% Confidence VaR']
gbm_values = [metrics['GBM_VaR_95'], metrics['GBM_VaR_99']]
jd_values = [metrics['JD_VaR_95'], metrics['JD_VaR_99']]

fig.add_trace(go.Bar(x=categories, y=gbm_values, marker_color='dodgerblue',
name='Pure Diffusion (GBM)'), row=2, col=1)
fig.add_trace(go.Bar(x=categories, y=jd_values, marker_color='crimson',
name='Jump Diffusion (Merton)'), row=2, col=1)

# Panel 4: Percentage Delta Underestimation
underestimation_95 = ((metrics['JD_VaR_95'] - metrics['GBM_VaR_95']) /
metrics['JD_VaR_95']) * 100
underestimation_99 = ((metrics['JD_VaR_99'] - metrics['GBM_VaR_99']) /
metrics['JD_VaR_99']) * 100

fig.add_trace(go.Bar(
    x=categories, y=[underestimation_95, underestimation_99],
    marker_color='orange', text=[f"{underestimation_95:.1f}%", f"
{underestimation_99:.1f}%"],
    textposition='auto', name='Underestimation %'
), row=2, col=2)

# Layout and Axis Configuration

```



```

fig.update_layout(
    title_text="Nomura Cash Equities CRB – Stochastic Jump-Diffusion Risk
Management Cockpit",
    title_x=0.5, height=900, width=1300, template="plotly_dark",
    showlegend=False
)

fig.update_xaxes(title_text="Trading Horizon Steps (Days)", row=1, col=1)
fig.update_yaxes(title_text="Asset Price Matrix Level", row=1, col=1)
fig.update_xaxes(title_text="Realized Log-Return Horizon Outcomes", row=1,
col=2)
fig.update_yaxes(title_text="Probability Density", row=1, col=2)
fig.update_yaxes(title_text="Calculated VaR Capital Charge Scale", row=2,
col=1)
fig.update_yaxes(title_text="Omission Risk Deficit Percentage (%)", row=2,
col=2)

fig.show()

# =====
# 4. EXECUTION GATEWAY MAIN CONTROL ENTRY POINT
# =====
if __name__ == "__main__":
    # Define structural parameters for testing
    # Captures an environment with a 1-year horizon, daily tracking, and frequent
    downward jumps
    initial_price = 100.0
    drift_rate = 0.06
    diffusion_vol = 0.18
    jump_intensity = 3.0          # Average of 3 sudden liquidity shocks per year
    expected_log_jump = -0.12     # Average jump size is a -12% price decline
    jump_vol_std = 0.04          # Variance around the jump size

    horizon_t = 1.0
    trading_steps = 252
    simulated_paths = 10000

    # 1. Execute the Monte Carlo simulation engine
    price_history, final_returns, expected_kappa = simulate_merton_jump_diffusion(
        S0=initial_price, mu=drift_rate, sigma=diffusion_vol,
        lam=jump_intensity, mu_J=expected_log_jump, delta=jump_vol_std,
        T=horizon_t, n_steps=trading_steps, n_paths=simulated_paths
    )

    # 2. Compute the comparative Value-at-Risk metrics
    risk_metrics = compute_comparative_var_metrics(final_returns, drift_rate,
diffusion_vol, horizon_t)

    print("\n" + "="*65)
    print("STOCHASTIC JUMP MODEL EXTRACTION PROFILE:")
    print("="*65)
    print(f"Calculated Jump Compensator Drift (Kappa) : {expected_kappa:.5f}")
    print(f"Pure Diffusion (GBM) 95% Confidence VaR :
{risk_metrics['GBM_VaR_95']:.5f}")

```

```

    print(f"Merton Jump Diffusion 95% Confidence VaR :
    {risk_metrics['JD_VaR_95']:.5f}")
    print(f"Pure Diffusion (GBM) 99% Confidence VaR :
    {risk_metrics['GBM_VaR_99']:.5f}")
    print(f"Merton Jump Diffusion 99% Confidence VaR :
    {risk_metrics['JD_VaR_99']:.5f}")
    print("-"*65)
    print(f"99% Confidence Risk Capital Deficit: {risk_metrics['JD_VaR_99'] -
    risk_metrics['GBM_VaR_99']:.5f}")
    print("="*65 + "\n")

# 3. Launch the visual platform
timeline_steps = np.arange(trading_steps + 1)
generate_merton_risk_dashboard(timeline_steps, price_history, final_returns,
risk_metrics)

```

3. Strategic Interview Follow-Ups & Edge-Case Vulnerabilities

Follow-Up 1: The Hedging Deficit in Jump-Diffusion (Incomplete Markets)

Question: "In a pure Black-Scholes continuous Brownian motion framework, markets are dynamically complete. This means any derivative payoff can be perfectly replicated using a continuous delta-hedging strategy. Does this completeness property hold under Merton's jump-diffusion architecture? What are the practical implications for managing options risk within a Central Risk Book?"

- **The Answer:** No, introducing discrete jumps transforms the model into an **incomplete market framework**.
- **The Reason:** A standard Brownian motion can be perfectly hedged using a single underlying asset because its changes are continuous and normally distributed over time. However, a jump process adds a source of uncertainty: the jump arrival time (N_t) and the random jump size (J_t).
- **The Consequence for the Book:** Since a single underlying stock cannot hedge a continuous spectrum of potential jump sizes, you cannot eliminate all risk using a standard delta-hedging approach. This residual risk is known as **jump risk**.
- To minimize this risk in a production environment, the central risk book must move beyond basic delta replication. It must implement a **mean-variance risk-minimization framework** or incorporate liquid out-of-the-money options directly into the hedging portfolio to neutralize the expected variance of the jump component.

Follow-Up 2: Calibrating the Jump Process via Option Implied Vol Skew

Question: "We want to calibrate our Merton model parameters ($\sigma, \lambda, \mu_J, \delta$) to current market data. If you calibrate the model using historical stock returns versus calibrating it to the current options implied volatility smile/skew surface, what structural differences do you expect to see in the parameters? Why do these differences occur?"

- **The Trap:** Answering that both calibration approaches should yield identical parameter values because they describe the same underlying asset. This response overlooks the difference between empirical and

risk-neutral distributions.

- **The Answer:** Calibrating the model to historical time-series returns provides parameters under the **Physical Probability Measure ($\mathbb{P}$)**. In contrast, calibrating to options implied volatilities provides parameters under the **Risk-Neutral Probability Measure ($\mathbb{Q}$)**.
- When calibrating to market options prices, you will typically find that the risk-neutral jump intensity ($\lambda^{\mathbb{Q}}$) is significantly higher than the historically observed frequency ($\lambda^{\mathbb{P}}$), and the risk-neutral expected jump size ($\mu_J^{\mathbb{Q}}$) is more deeply negative than the empirical average ($\mu_J^{\mathbb{P}}$).
- **The Economic Reason:** This difference reflects the **variance and jump risk premium** demanded by the market. Institutional investors buy out-of-the-money put options to hedge against sudden market sell-offs. This heavy demand bids up the prices of these options, causing the implied volatility skew to reflect a higher probability of extreme downside events than has historically occurred. The calibrated risk-neutral parameters adjust to reflect this market risk premium.